

Fullstack Developer Test:

Aplikasi "Simple Task Management Board"

Tech Stack: Express.js, Sequelize, PostgreSQL, React.js, Vite, Docker.

Due Date: Jumat, 7 November 2025 pukul 23.59 WIB

1. Deskripsi Tugas

Buatlah sebuah aplikasi *full-stack* untuk manajemen tugas sederhana (mirip Trello/Kanban board sederhana) di mana pengguna dapat membuat, melihat, memperbarui, dan menghapus tugas.

Aplikasi ini harus terdiri dari Backend REST API, Frontend berbasis React, dan dikonfigurasi untuk berjalan sepenuhnya menggunakan Docker.

2. Backend (Express.js, Sequelize, PostgreSQL)

Buat REST API dengan struktur yang rapi (misal: memisahkan routes, controllers, models).

Model Task (PostgreSQL/Sequelize)

- id: UUID (atau integer)
- title: String (wajib diisi, tidak boleh kosong)
- description: Text (opsional)
- status: Enum/String (Kolom ini wajib, dengan nilai default: 'To Do'. Nilai yang diizinkan: 'To Do', 'In Progress', 'Done')
- createdAt, updatedAt: (dibuat otomatis oleh Sequelize)

Endpoints API

1. **POST /api/tasks:** Membuat tugas baru.
 - Input: { "title": "...", "description": "..." }

- status harus otomatis diatur ke 'To Do' saat pembuatan.
- Terapkan validasi: title tidak boleh kosong.

2. **GET /api/tasks:** Mendapatkan semua tugas.

- Respons harus mengelompokkan tugas berdasarkan statusnya agar mudah dirender oleh frontend.

- Contoh Respons:

JSON

```
{
  "To Do": [
    { "id": 1, "title": "Task A", ... },
    { "id": 2, "title": "Task B", ... }
  ],
  "In Progress": [
    { "id": 3, "title": "Task C", ... }
  ],
  "Done": [
    { "id": 4, "title": "Task D", ... }
  ]
}
```

3. **PUT /api/tasks/:id:** Memperbarui tugas.

- Bisa digunakan untuk memperbarui title, description, atau status (untuk memindahkan tugas antar kolom).

4. **DELETE /api/tasks/:id:** Menghapus tugas.

Persyaratan Backend

- **Migrations:** *Setup* tabel database *harus* menggunakan Sequelize Migrations.
- **Validasi:** Terapkan validasi input (misal: title tidak boleh kosong) menggunakan express-validator atau validasi bawaan Sequelize.
- **Error Handling:** Buat satu *middleware* error handling terpusat untuk menangani kesalahan secara konsisten.
- **Struktur:** Kode harus terorganisir dengan baik.

3. Frontend (React.js, Vite)

Buat antarmuka (UI) untuk menampilkan *board* Kanban menggunakan React.js dan Vite.

Fitur UI

1. Tampilan Board:

- Tampilkan 3 kolom: "To Do", "In Progress", "Done".
- Render *card* tugas di bawah kolom yang sesuai berdasarkan data yang diterima dari API (GET /api/tasks).

2. Tambah Tugas:

- Buat tombol "Tambah Tugas" yang membuka *modal* atau *form*.
- Form ini mengirim data ke POST /api/tasks.
- Setelah berhasil, *board* harus otomatis diperbarui untuk menampilkan tugas baru di kolom "To Do".

3. Update Status (Pindah Kolom):

- Pengguna harus bisa mengubah status tugas untuk memindahkannya ke kolom lain.
- **Opsi 1 (Wajib):** Setiap *card* tugas memiliki *dropdown* untuk memilih status baru (To Do, In Progress, Done).
- **Opsi 2 (Bonus):** Implementasi *Drag and Drop* (misal: react-beautiful-dnd atau dnd-kit) untuk memindahkan *card* antar kolom.
- Aksi ini harus memanggil PUT /api/tasks/:id dengan status yang baru.

4. Hapus Tugas:

- Setiap *card* memiliki tombol hapus (disarankan ada konfirmasi) yang memanggil DELETE /api/tasks/:id.

Persyaratan Frontend

- Gunakan React Hooks (terutama useState, useEffect) untuk manajemen *state* dan *logic*.

- Gunakan axios atau fetch untuk berkomunikasi dengan API backend.
- Pisahkan UI menjadi komponen-komponen yang *reusable* (misal: <Board>, <Column>, <TaskCard>).

4. Infrastruktur (Docker)

Kandidat harus menyediakan konfigurasi Docker agar keseluruhan aplikasi (Backend, Frontend, DB) dapat berjalan dengan satu perintah (docker-compose up).

1. Dockerfile (Backend):

- Buat Dockerfile yang efisien untuk aplikasi Express.js.

2. Dockerfile (Frontend):

- Buat Dockerfile untuk aplikasi React.js.
- **Nilai Plus (Mid-level):** Menggunakan *multi-stage build* (Stage 1: node untuk *build* aset statis, Stage 2: nginx untuk *serve* hasil *build* tersebut).

3. docker-compose.yml:

- Buat file docker-compose.yml yang mendefinisikan 3 *services*:
 - backend: Menjalankan aplikasi Express.js.
 - frontend: Menjalankan aplikasi React/Nginx.
 - db: Menjalankan *image* postgres:latest.
- Konfigurasi *environment variables* (untuk koneksi DB, port, dll). Jangan *hardcode* kredensial.
- Konfigurasi *volumes* untuk *database persistence* (data di db tidak boleh hilang saat *container restart*).
- Pastikan *service* backend menunggu db siap sebelum *start* (misal: menggunakan *depends_on*).

5. Instruksi Pengumpulan & Dokumentasi

- Upload seluruh kode (backend, frontend, file Docker) ke repositori Git (GitHub/GitLab).
- Buat file README.md yang jelas, berisi:
 - Penjelasan singkat aplikasi.

- Daftar *environment variables* yang diperlukan (sertakan file `.env.example`).
- Instruksi cara menjalankan proyek **HANYA** dengan satu perintah: `docker-compose up --build`.

Pengiriman

Kumpulkan ke `hermawan.fk@kalachakra.io`

Dengan subject: **Tes Fullstack Developer - [Nama]**

Sertakan: **Link repository GitHub/GitLab**